

ASSIGNMENT

Keys & Constraints

Q1. Design the departments Table.

Objective:

The objective of this question is to create a departments master table and understand how Primary Key, NOT NULL, and UNIQUE constraints maintain the accuracy and uniqueness of department data.

Theory:

A Primary Key uniquely identifies every department in the table. It cannot contain duplicate or NULL values.

A NOT NULL constraint ensures that every department must have a name.

A UNIQUE constraint prevents two departments from having the same name.

For example:

dept_id = 1 → Computer Science

dept_id = 2 → Mechanical Engineering

The dept_id uniquely identifies each department, while dept_name cannot be duplicated.

SQL Solution:

a. Create table

```
1 CREATE TABLE academics.departments (  
2   dept_id INTEGER PRIMARY KEY,  
3   dept_name VARCHAR(100) NOT NULL UNIQUE  
4 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 203 msec.

b. Insert Data

```
6 SELECT * from academics.departments;  
7 INSERT INTO academics.departments (dept_id, dept_name)  
8 VALUES  
9 (1, 'Computer Science'),  
10 (2, 'Mechanical Engineering'),  
11 (3, 'Electronics Engineering'),  
12 (4, 'Civil Engineering');  
13
```

Data Output Messages Notifications

Showing rows: 1 to 4 of 1

dept_id [PK] Integer	dept_name character varying (100)
1	Computer Science
2	Mechanical Engineering
3	Electronics Engineering
4	Civil Engineering

Q2. Retrofit a Foreign Key

Objective:

The objective is to improve the existing students table by replacing the unreliable text-based department relationship with a proper Foreign Key relationship.

Theory:

Currently, the students table contains:

department VARCHAR

This allows spelling mistakes such as:

Computer Science

Computr Science

Computer Scince

These are treated as different values.

A Foreign Key solves this problem by storing the department's ID instead.

For example:

students.dept_id



departments.dept_id

The value entered into students.dept_id must already exist in departments.dept_id.

This maintains referential integrity.

SQL Solution:

a. Add dept_id

```
14
15 ALTER TABLE academics.students
16 ADD COLUMN dept_id INTEGER;
17 SELECT * FROM academics.students;
```

		enrolled_at timestamp with time zone	phone_number character varying (15)	department character varying (50)	dept_id integer
1	se": "BE Computer Science")	2023-08-18 09:45:00+00	9988776655	Computer Science	[null]
2	se": "BE Computer Science")	2023-08-15 09:30:00+00	9876543210	Computer Science	[null]
3	4, "course": "BE Computer Scien...	2023-08-16 10:15:00+00	9812345678	Computer Science	[null]
4	ourse": "BE Data Science")	2023-08-17 11:00:00+00	9123456789	Computer Science	[null]

b. Populate dept_id from Existing Department Names

```
19 UPDATE academics.students s
20 SET dept_id = d.dept_id
21 FROM academics.departments d
22 WHERE s.department = d.dept_name;
```

UPDATE	4
Query returned successfully in 223 msec.	

c. Add Foreign Key

```
24 ALTER TABLE academics.students
25 ADD CONSTRAINT fk_students_department
26 FOREIGN KEY (dept_id)
27 REFERENCES academics.departments(dept_id);
28
```

ALTER TABLE	
Query returned successfully in 78 msec.	

d. Verify

```
29 SELECT student_id, student_name, department, dept_id
30 FROM academics.students;
31
```

Data Output Messages Notifications

Showing rows: 1 to 4 of 1

	student_id [PK] integer	student_name character varying (100)	department character varying (50)	dept_id integer
1	3	Priya Verma	Computer Science	1
2	2	Reetiks	Computer Science	1
3	1	Sunakshi	Computer Science	1
4	4	Vanshika	Computer Science	1

Q3. Composite Primary Key Junction Table

Objective:

The objective is to create an enrollments junction table that represents the many-to-many relationship between students and courses while preventing duplicate enrollment.

Theory:

A student can enroll in many courses, and one course can have many students.

Therefore:

Students $\leftrightarrow$ Courses

is a many-to-many relationship.

A junction table is used:

students



enrollments



courses

The combination of: (student_id, course_id) forms a Composite Primary Key.

This means the same student cannot be enrolled in the same course twice.

For example:

Student 1 + Course 101 $\rightarrow$ valid

Student 1 + Course 102 $\rightarrow$ valid

Student 2 + Course 101 $\rightarrow$ valid

Student 1 + Course 101 $\rightarrow$ duplicate $\rightarrow$ invalid

SQL Solution:

```
33 CREATE TABLE academics.enrollments (  
34     student_id INTEGER,  
35     course_id INTEGER,  
36     enroll_date DATE DEFAULT CURRENT_DATE,  
37     grade CHAR(2),  
38  
39     PRIMARY KEY (student_id, course_id),  
40  
41     FOREIGN KEY (student_id)  
42         REFERENCES academics.students(student_id),  
43  
44     FOREIGN KEY (course_id)  
45         REFERENCES academics.courses(course_id)  
46 );
```

Data Output Messages Notifications

student_id	course_id	enroll_date	grade
[PK] integer	[PK] integer	date	character (2)

Q4. Enforce a Valid Grade Range

Objective:

The objective is to ensure that only valid academic grades can be stored in the enrollments table.

Theory:

A CHECK constraint restricts the values that can be stored in a column based on a condition.

The college allows: A+, A, B+, B, C, F, NULL

Any other value should be rejected.

For example:

A+ → valid

B → valid

D → invalid

X → invalid

The constraint should be explicitly named so that it is easier to identify when an error occurs.

SQL Solution:

```
50 ALTER TABLE academics.enrollments  
51 ADD CONSTRAINT chk_enrollments_grade  
52 CHECK (  
53     grade IN ('A+', 'A', 'B+', 'B', 'C', 'F')  
54 );  
55  
56 SELECT conname, pg_get_constraintdef(oid)  
57 FROM pg_constraint  
58 WHERE conrelid = 'academics.enrollments'::regclass;
```

Data Output Messages Notifications

conname	pg_get_constraintdef
name	text
1	enrollments_pkey
2	enrollments_student_id_fk...
3	enrollments_course_id_fk...
4	chk_enrollments_grade

Q5. CASCADE Delete Behavior

Objective:

The objective is to understand and implement ON DELETE CASCADE so that deleting a student automatically removes that student's enrollment records.

Theory:

ON DELETE CASCADE means:

When a parent record is deleted, its related child records are automatically deleted.

Here:

students
↓
enrollments

students is the parent table and enrollments is the child table.

If Ravi Singh is deleted:

Ravi Singh
↓
Enrollment 1 → deleted
Enrollment 2 → deleted

This prevents orphan records.

SQL Solution:

a. Configure CASCADE

If the existing Foreign Key exists, remove it first:

```
60
61 ALTER TABLE academics.enrollments
62 DROP CONSTRAINT enrollments_student_id_fkey;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 333 msec.

Then

The screenshot shows a database management tool interface. On the left, a tree view displays the database structure, including 'departments', 'enrollments', 'Columns (4)', 'Constraints (4)', 'Indexes', 'RLS Policies', 'Rules', and 'Triggers'. The 'enrollments' table is selected, and its constraints are visible: 'chk_enrollments', 'enrollments_cou', 'enrollments_pke', and 'fk_enrollments_s'. On the right, the SQL editor shows the following command:

```
63
64 ALTER TABLE academics.enrollments
65 ADD CONSTRAINT fk_enrollments_student
66 FOREIGN KEY (student_id)
67 REFERENCES academics.students(student_id)
68 ON DELETE CASCADE;
```

Below the SQL editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. At the bottom, a table structure is displayed with columns: 'student_id' (integer, PK), 'course_id' (integer, PK), 'enroll_date' (date), and 'grade' (character (2)).

b. Test it

• Insert Student

```
69
70 INSERT INTO academics.students
71 (student_id, student_name, email, dept_id)
72 VALUES
73 (9998, 'Ravi Singh', 'ravi9998@example.com', 1);
74 SELECT * FROM academics.students;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	student_id [PK] integer	student_name character varying (100)	dob date	email text	cgpa numeric (4,2)	is_active boolean	skills text[]	profile jsonb
1	3	Priya Verma	2004-02-...	priya.verma@gmail.c...	9.12	true	{Python,'Machine Learning',Pand...	{'city': 'Shimla', 'year': 4, 'course': 'BE Data Science'
2	2	Reetiks	2003-11-...	reetika@gmail.com	8.65	true	{Java,'Spring Boot',SQL}	{'city': 'Chandigarh', 'year': 4, 'course': 'BE Compute
3	1	Sunakshi	2004-06-...	sunakshi@gmail.com	8.90	true	{C#,SQL,React}	{'city': 'Una', 'year': 4, 'course': 'BE Computer Scien
4	4	Vanshika	2003-09-...	vanshika@gmail.com	7.89	true	{C++,DSA,Git}	{'city': 'Delhi', 'year': 4, 'course': 'BE Computer Scier
5	9998	Ravi Singh	[null]	ravi9998@example.co...	[null]	true	[null]	[null]

• Insert Two Enrollments

```
76 INSERT INTO academics.enrollments
77 (student_id, course_id, grade)
78 VALUES
79 (9998, 1, 'A'),
80 (9998, 2, 'B+');
81 SELECT * FROM academics.enrollments;
```

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	enroll_date date	grade character (2)
1	9998	1	2026-09-07	A
2	9998	2	2026-09-07	B+

• Check Before Delete

```
83 SELECT *
84 FROM academics.enrollments
85 WHERE student_id = 9998;
```

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	enroll_date date	grade character (2)
1	9998	1	2026-09-07	A
2	9998	2	2026-09-07	B+

• Delete Student

```
87 DELETE FROM academics.students
88 WHERE student_id = 9998;
89
90 SELECT *
91 FROM academics.enrollments
92 WHERE student_id = 9998;
```

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	enroll_date date	grade character (2)
--	----------------------------	---------------------------	---------------------	------------------------

This proves that ON DELETE CASCADE worked.

Q6. RESTRICT vs CASCADE on courses

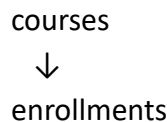
Objective:

The objective is to prevent deletion of a course when students are still enrolled in it and understand why RESTRICT is safer for academic records.

Theory:

ON DELETE RESTRICT prevents deletion of a parent record if child records reference it.

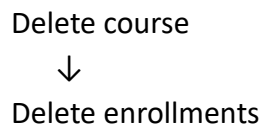
Here:



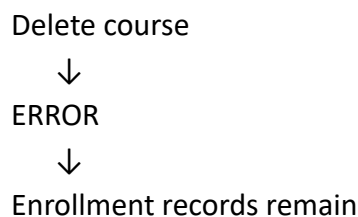
If students are enrolled in Course 1, Course 1 cannot be deleted.

This is different from CASCADE.

CASCADE



RESTRICT



For academic systems, RESTRICT protects historical enrollment information.

SQL Solution:

a. Configure Restrict

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Constraints (3)' folder is expanded, showing 'chk_enrollments', 'enrollments_pkey', and 'fk_enrollments_course'. The 'Messages' tab is selected in the bottom right pane. The SQL query editor shows the following code:

```
ALTER TABLE academics.enrollments
DROP CONSTRAINT enrollments_course_id_fkey;
```

The message pane displays: 'Query returned successfully in 115 msec.'

Then

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'enrollments' table is selected, and the 'Constraints (4)' folder is expanded, showing 'chk_enrollments_grade', 'enrollments_pkey', 'fk_enrollments_course', and 'fk_enrollments_student'. The 'Messages' tab is selected in the bottom right pane. The SQL query editor shows the following code:

```
ALTER TABLE academics.enrollments
ADD CONSTRAINT fk_enrollments_course
FOREIGN KEY (course_id)
REFERENCES academics.courses(course_id)
ON DELETE RESTRICT;
```

The message pane displays: 'Query returned successfully in 73 msec.'

b. Attempt Delete

```
106 DELETE FROM academics.courses
107 WHERE course_id = 1;
108
109 SELECT *
110 FROM academics.enrollments
111 WHERE course_id = 1;
112
113 SELECT *
114 FROM academics.courses
115 WHERE course_id = 1;
116
117 INSERT INTO academics.enrollments
118 (student_id, course_id, grade)
119 VALUES
120 (1, 1, 'A');
121
```

Data Output Messages Notifications

ERROR: insert or update on table "enrollments" violates foreign key constraint "fk_enrollments_course"
Key (course_id)=(1) is not present in table "courses".

SQL state: 23503

Detail: Key (course_id)=(1) is not present in table "courses".

c. Why RESTRICT?

RESTRICT prevents accidental deletion of a course that is still referenced by enrollment records. Unlike CASCADE, it preserves academic history and prevents related enrollment data from being automatically removed.

Q7. UNIQUE Constraint with a Real-World Twist

Objective:

The objective is to ensure that every student has a unique email address while allowing multiple students to have the same name.

Theory:

A UNIQUE constraint prevents duplicate values in a column.

Student names do not need to be unique because two students may legitimately have the same name.

For example:

Rahul Sharma → rahul1@gmail.com

Rahul Sharma → rahul2@gmail.com

is valid.

But:

Rahul Sharma → abc@gmail.com

Aman Kumar → abc@gmail.com

is invalid because the email address is duplicated.

SQL Solution:

- Add UNIQUE Constraint

The screenshot shows the 'students' table structure in SQL Developer. The 'Constraints' tab is selected, showing three constraints: 'fk_students_department', 'students_pkey', and 'uq_students_email'. The 'Messages' tab is also visible, showing an error message: 'ERROR: relation "uq_students_email" already exists' and 'SQL state: 42P07'.

```
123 ALTER TABLE academics.students
124 ADD CONSTRAINT uq_students_email
125 UNIQUE (email);
126
```

Data Output Messages Notifications

ERROR: relation "uq_students_email" already exists

SQL state: 42P07

- Check Existing Email

The screenshot shows a SQL query in SQL Developer: 'SELECT student_id, student_name, email FROM academics.students LIMIT 1;'. The 'Data Output' tab is selected, showing the results of the query. The results are displayed in a table with columns 'student_id', 'student_name', and 'email'. The first row shows '1', 'Priya Verma', and 'priya.verma@gmail.c...'.

```
127 SELECT student_id, student_name, email
128 FROM academics.students
129 LIMIT 1;
130
```

Data Output Messages Notifications

student_id	student_name	email
1	Priya Verma	priya.verma@gmail.c...

- Then try to insert with existing email

The screenshot shows a SQL query in SQL Developer: 'INSERT INTO academics.students (student_id, student_name, email, dept_id) VALUES (9997, 'Anu Priya', 'priya.verma@gmail.com', 1);'. The 'Messages' tab is selected, showing an error message: 'ERROR: duplicate key value violates unique constraint "uq_students_email" Key (email)=(priya.verma@gmail.com) already exists.' and 'SQL state: 23505'.

```
131 INSERT INTO academics.students
132 (student_id, student_name, email, dept_id)
133 VALUES
134 (9997, 'Anu Priya', 'priya.verma@gmail.com', 1);
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "uq_students_email"
Key (email)=(priya.verma@gmail.com) already exists.

SQL state: 23505
Detail: Key (email)=(priya.verma@gmail.com) already exists.

Q8. SET NULL on Optional Relationship

Objective:

The objective is to create an optional advisor relationship and understand how ON DELETE SET NULL preserves student records when an advisor is deleted.

Theory:

Sometimes a relationship is optional.

A student can have:

advisor_id = 1

or: advisor_id = NULL

If the advisor leaves the college, we do not want to delete the student.

Instead:

Advisor deleted



Student remains



advisor_id = NULL

ON DELETE SET NULL performs exactly this operation.

SQL Solution:

a. Create Advisors

```
136 CREATE TABLE academics.advisors (  
137     advisor_id INTEGER PRIMARY KEY,  
138     advisor_name VARCHAR(100) NOT NULL  
139 );  
140  
141 SELECT * from academics.advisors;
```

Data Output Messages Notifications

	advisor_id [PK] integer	advisor_name character varying (100)
--	----------------------------	---

Insert Advisors

```
144 INSERT INTO academics.advisors  
145 (advisor_id, advisor_name)  
146 VALUES  
147 (1, 'Dr. Mehta'),  
148 (2, 'Dr. Sharma'),  
149 (3, 'Dr. Kapoor');
```

Data Output Messages Notifications

	advisor_id [PK] integer	advisor_name character varying (100)
1	1	Dr. Mehta
2	2	Dr. Sharma
3	3	Dr. Kapoor

b. Add Advisor to Students

```
152 ALTER TABLE academics.students  
153 ADD COLUMN advisor_id INTEGER;  
154
```

Data Output Messages Notifications

Showing rows: 1 to 4								Page No: 1 of 1				
	is	profile	enrolled_at	phone_number	department	dept_id	advisor_id					
1	thon,"Machine Learning",Pand...	{'city': 'Shimla', 'year': 4, 'course': 'BE Data Science'}	2023-08-17 11:00:00+00	9123456789	Computer Science	1	[null]					
2	va,"Spring Boot",SQL	{'city': 'Chandigarh', 'year': 4, 'course': 'BE Computer Scien...	2023-08-16 10:15:00+00	9812345678	Computer Science	1	[null]					
3	SQL,React	{'city': 'Una', 'year': 4, 'course': 'BE Computer Science'}	2023-08-15 09:30:00+00	9876543210	Computer Science	1	[null]					
4	+DSA,Git	{'city': 'Delhi', 'year': 4, 'course': 'BE Computer Science'}	2023-08-18 09:45:00+00	9988776655	Computer Science	1	[null]					

Add a foreign key:

students

Columns

Constraints (4)

- fk_students_advisor
- fk_students_department
- students_pkey
- uq_students_email

Indexes

RLS Policies

```
154
155 ALTER TABLE academics.students
156 ADD CONSTRAINT fk_students_advisor
157 FOREIGN KEY (advisor_id)
158 REFERENCES academics.advisors(advisor_id)
159 ON DELETE SET NULL;
160
```

Data Output Messages Notifications

ERROR: constraint "fk_students_advisor" for relation "students" already exists

SQL state: 42710

c. Test

Assign advisor and check

```
161 UPDATE academics.students
162 SET advisor_id = 1
163 WHERE student_id = 1;
164
165 SELECT student_id, student_name, advisor_id
166 FROM academics.students
167 WHERE student_id = 1;
```

Data Output Messages Notifications

	student_id [PK] integer	student_name character varying (100)	advisor_id integer
1	1	Sunakshi	1

Delete advisor and verify

```
169 DELETE FROM academics.advisors
170 WHERE advisor_id = 1;
171
172 SELECT student_id, student_name, advisor_id
173 FROM academics.students
174 WHERE student_id = 1;
```

Data Output Messages Notifications

	student_id [PK] integer	student_name character varying (100)	advisor_id integer
1	1	Sunakshi	[null]

Q9. Composite UNIQUE Constraint

Objective:

The objective is to ensure that a course name is unique within a department, while allowing the same course name in different departments.

Theory:

A normal: `UNIQUE(course_name)` would prevent:

Computer Science → Database

Mechanical → Database

But the college wants this to be allowed.

Therefore, uniqueness must be based on the combination:

`dept_id + course_name`

This is called a Composite UNIQUE Constraint.

Valid

1 + Database

2 + Database

because the combinations are different.

Invalid

1 + Database

1 + Database

because the complete combination is duplicated.

SQL Solution:

```
198 SELECT *
199 FROM academics.courses;
200
201 ALTER TABLE academics.courses
202 ADD CONSTRAINT uq_courses_department_name
203 UNIQUE (dept_id, course_name);
204
```

Data Output Messages Notifications

	course_id [PK] integer	course_name character varying (100)	credits smallint	department character varying (50)	dept_id integer
1	2	Database Management Syste...	4	Computer Science	1
2	3	Computer Networks	3	Information Technology	2
3	4	Operating Systems	4	Computer Science	2
4	100	Database	[null]	[null]	2
5	101	Database	[null]	[null]	1

Check

```
205 INSERT INTO academics.courses
206 (course_id, course_name, dept_id)
207 VALUES
208 (100, 'Database', 2);
209
210 INSERT INTO academics.courses
211 (course_id, course_name, dept_id)
212 VALUES
213 (101, 'Database', 1);
214
215
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "courses_pkey"
Key (course_id)=(100) already exists.

SQL state: 23505
Detail: Key (course_id)=(100) already exists.

Q10. Debugging a Broken Insert

Objective:

The objective is to understand how Foreign Key constraints protect the database from inserting records that reference non-existent students.

Theory:

The given statement is:

```
INSERT INTO academics.enrollments
(student_id, course_id, grade)
VALUES
(9999, 1, 'A+');
```

enrollments.student_id is a Foreign Key referencing: students.student_id

Therefore, student 9999 must already exist.

If it does not exist, PostgreSQL rejects the insertion.

This is called a Foreign Key violation.

SQL Solution:

Check Student, if doesn't exist insert student info

```
216 SELECT *
217 FROM academics.students
218 WHERE student_id = 9999;
219
220 INSERT INTO academics.students
221 (student_id, student_name, email, dept_id)
222 VALUES
223 (9999, 'Test Student', 'test9999@example.com', 1);
224
```

Data Output Messages Notifications								
Showing rows: 1 to 1								
	student_id [PK] integer	student_name character varying (100)	dob date	email text	cgpa numeric (4,2)	is_active boolean	skills text[]	profile jsonb
1	9999	Test Student	[null]	test9999@example.c...	[null]	true	[null]	[null]

Then execute

```
225 INSERT INTO academics.enrollments
226 (student_id, course_id, grade)
227 VALUES
228 (9999, 1, 'A+');
```

Data Output Messages Notifications

ERROR: insert or update on table "enrollments" violates foreign key constraint "fk_enrollments_course"
Key (course_id)=(1) is not present in table "courses".

SQL state: 23503

Detail: Key (course_id)=(1) is not present in table "courses".

Why Did the Original Statement Fail?

Because 9999 → students.student_id did not exist.

The Foreign Key constraint prevents an enrollment from referring to a non-existent student.

Q11. CHECK Constraint with Multiple Conditions

Objective:

The objective is to ensure that student data satisfies two conditions simultaneously:

1. CGPA must be between 0 and 10.
2. Age must be at least 16.

Theory:

A CHECK constraint validates data before it is inserted or updated.

Here we need two conditions:

CGPA between 0 and 10

AND

Age >= 16

The AND operator means both conditions must be satisfied.

Valid

CGPA = 8.5

Age = 20

Invalid

CGPA = 11

Age = 20

because CGPA is greater than 10.

Also invalid:

CGPA = 8.5

Age = 14

because age is less than 16.

SQL Solution:

- **Add Age**

```
231 ALTER TABLE academics.students
232 ADD COLUMN age SMALLINT;
233
234 SELECT * from academics.students;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	profile jsonb	enrolled_at timestamp with time zone	phone_number character varying (15)	department character varying (50)	dept_id integer	advisor_id integer	age smallint
1	{\"city\": \"Shimla\", \"year\": 4, \"course\": \"BE Data Science\"}	2023-08-17 11:00:00+00	9123456789	Computer Science	1	[null]	[null]
2	{\"city\": \"Chandigarh\", \"year\": 4, \"course\": \"BE Computer Scien...\"}	2023-08-16 10:15:00+00	9812345678	Computer Science	1	[null]	[null]
3	{\"city\": \"Delhi\", \"year\": 4, \"course\": \"BE Computer Science\"}	2023-08-18 09:45:00+00	9988776655	Computer Science	1	[null]	[null]
4	{\"city\": \"Una\", \"year\": 4, \"course\": \"BE Computer Science\"}	2023-08-15 09:30:00+00	9876543210	Computer Science	1	[null]	[null]
5	[null]	2026-09-07 05:53:45.544637+...	[null]	[null]	1	[null]	[null]

- **Add the Check Constraint**

students

Columns

Constraints (5)

chk_students_cgpa_age

fk_students_advisor

fk_students_department

students_pkey

uq_students_email

Indexes

RLS Policies

```
236 ALTER TABLE academics.students
237 ADD CONSTRAINT chk_students_cgpa_age
238 CHECK (
239     cgpa BETWEEN 0 AND 10
240     AND age >= 16
241 );
242
243
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 91 msec.

- **Check Existing Invalid Data**

```

243 SELECT student_id, student_name, cgpa, age
244 FROM academics.students
245 WHERE cgpa NOT BETWEEN 0 AND 10
246        OR age < 16;
247
248 INSERT INTO academics.students
249 (student_id, student_name, email, cgpa, age)
250 VALUES
251 (2002, 'Aman Kumar', 'aman2002@gmail.com', 11.5, 20);
252
253

```

Data Output Messages Notifications

ERROR: new row for relation "students" violates check constraint "chk_students_cgpa_age"
 Failing row contains (2002, Aman Kumar, null, aman2002@gmail.com, 11.50, t, null, null, 2026-09-07 06:11:18.178065+00, null, null, null, null, 20).

SQL state: 23514
 Detail: Failing row contains (2002, Aman Kumar, null, aman2002@gmail.com, 11.50, t, null, null, 2026-09-07 06:11:18.178065+00, null, null, null, null, 20).

Q12. Self-Referencing Foreign Key

Objective:

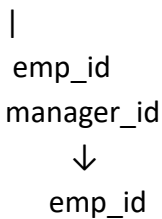
The objective is to create an employee hierarchy using a self-referencing Foreign Key and retrieve the reporting structure using a self-join.

Theory:

A self-referencing Foreign Key occurs when a table references its own Primary Key.

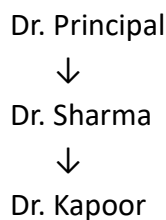
For example:

employees



An employee's manager_id refers to another employee's emp_id.

Example:



Here:

```

Dr. Principal → manager = NULL
Dr. Sharma   → manager = Dr. Principal
Dr. Kapoor   → manager = Dr. Sharma

```

The top-level employee has no manager, so manager_id is allowed to be NULL.

A self-join allows us to display an employee together with their manager.

SQL Solution:

a. Create Employees Table

```
253 CREATE TABLE academics.employees (  
254     emp_id INTEGER PRIMARY KEY,  
255     manager_id INTEGER,  
256     emp_name VARCHAR(100) NOT NULL,  
257  
258     CONSTRAINT fk_employee_manager  
259     FOREIGN KEY (manager_id)  
260     REFERENCES academics.employees(emp_id)  
261 );  
262
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 68 msec.

b. Insert Principal

```
263 INSERT INTO academics.employees  
264 (emp_id, manager_id, emp_name)  
265 VALUES  
266 (1, NULL, 'Dr. Principal');  
267  
268 SELECT * from academics.employees;  
269  
270  
271
```

Data Output Messages Notifications

	emp_id [PK] integer	manager_id integer	emp_name character varying (100)
1	1	[null]	Dr. Principal

c. Insert HOD

```
270 INSERT INTO academics.employees  
271 (emp_id, manager_id, emp_name)  
272 VALUES  
273 (2, 1, 'Dr. Sharma');  
274  
275
```

Data Output Messages Notifications

	emp_id [PK] integer	manager_id integer	emp_name character varying (100)
1	1	[null]	Dr. Principal
2	2	1	Dr. Sharma

d. Insert Professor

```
275 INSERT INTO academics.employees  
276 (emp_id, manager_id, emp_name)  
277 VALUES  
278 (3, 2, 'Dr. Kapoor');  
279
```

Data Output Messages Notifications

	emp_id [PK] integer	manager_id integer	emp_name character varying (100)
1	1	[null]	Dr. Principal
2	2	1	Dr. Sharma
3	3	2	Dr. Kapoor

e. Self-Join

```
280 SELECT
281     e.emp_id,
282     e.emp_name AS employee_name,
283     m.emp_name AS manager_name
284 FROM academics.employees e
285 LEFT JOIN academics.employees m
286     ON e.manager_id = m.emp_id
287 ORDER BY e.emp_id;
288
```

Data Output Messages Notifications

	emp_id [PK] integer	employee_name character varying (100)	manager_name character varying (100)
1	1	Dr. Principal	[null]
2	2	Dr. Sharma	Dr. Principal
3	3	Dr. Kapoor	Dr. Sharma